

Index Strategy × Workload

查询能力来自被维护的冗余，选择必须由操作分布决定

408 · 数据结构 Internal Bridge B02

桥梁接口

Workload → Required Operations → Maintained Order/Redundancy → Search/Update/Space/I/O Cost → Chosen Index

有序索引维护比较关系，Hash 维护槽位映射；二者不是“谁更快”的绝对排名，而是用不同冗余换取不同查询和更新能力。

Owners 与停止边界

DS09 Owns 折半查找、BST/AVL/B/B+ 机制；DS10 Owns Hash 冲突与装填；Atlas Foundation Owns cost vector。本桥不重复这些机制，只定义比较维度与选型证据。

目录

1	先把 workload 写成操作向量	2
2	冗余的交换关系	2
3	比较表与边界	2
4	做题控制协议	2

1 先把 workload 写成操作向量

记录精确等值、范围、前驱后继、排序遍历、插入删除更新的频率，以及是否要求最坏保证、稳定顺序、内存上限或块 I/O。若只问等值存在性且更新频繁，Hash 的平均成本可能占优；若要范围扫描或有序输出，有序索引保留必要顺序。

2 冗余的交换关系

维护排序或平衡信息会增加插入/删除修复；维护 Hash 槽位会支付冲突、装填和重散列。总成本可写成

$$C_{total} = \sum_i f_i C_{query,i} + \sum_j g_j C_{update,j} + C_{space} + C_{I/O}.$$

这不是精确机器计时，而是防止只比较一个操作的决策框架。

同一数据的两种选择

日志按 user_id 精确查找且很少范围查询：Hash 避免维护全序；若要求按时间区间扫描并支持前驱/后继，则需要有序索引或额外排序结构。workload 一变，Owner 不变，选择改变。

3 比较表与边界

策略	擅长	主要付费
折半/有序数组	静态查找、范围	插入移动、连续空间
BST/平衡树	动态有序查询	旋转、节点空间
Hash	精确等值	冲突、装填、重散列、无序
B/B+	大规模块索引、范围	分裂合并与 I/O

4 做题控制协议

先写至少两种候选，再逐项填查询、更新、空间、最坏与 I/O；任何“更快”都必须附带前提。若题目只给一个操作而没有 workload，保留多解并指出缺失信息，不擅自宣布唯一最佳结构。

压缩复原

四问：要查什么？多久更新？需不需要顺序？成本由时间、空间还是 I/O 主导？